From the Model to Samples

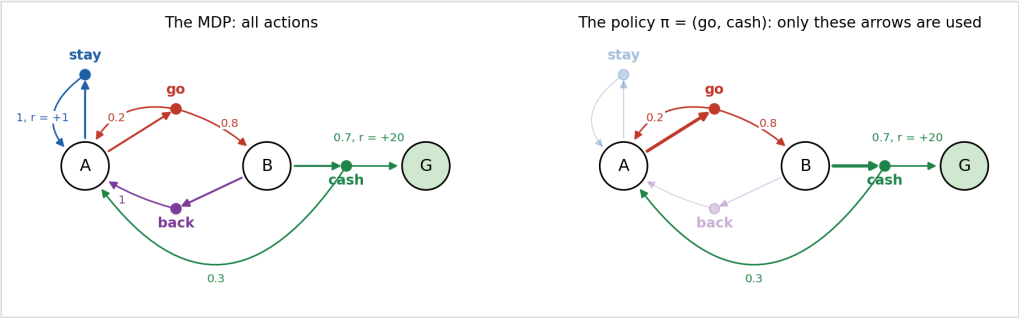
Evaluation and control on one toy MDP: exact solution, simple iterations, MC, TD, SARSA and Q-learning

Anton Selitskiy

ESE 561 · Theory of Artificial Intelligence

One toy MDP

The toy MDP



- States A, B, terminal G with $V(G) = 0$; discount $\gamma = 0.9$.
- A: **stay** ($+1$, remain in A) or **go** (to B w.p. 0.8, slip back w.p. 0.2).
- B: **cash** ($+20$ and finish w.p. 0.7, else back to A) or **back** (to A).
- We first evaluate the fixed policy $\pi = (\text{go}, \text{cash})$, then learn to control.

Roadmap: the same answer, six ways

	method	needs P?	idea
evaluate π	inverse matrix	yes	solve $V = r^\pi + \gamma P^\pi V$ exactly
	simple iterations	yes	$V_{k+1} = T^\pi V_k$, a γ-contraction
	Monte Carlo	no	average sampled returns G_t
	TD(0)	no	sampled step of $V \leftarrow T^\pi V$
control	SARSA, Expected SARSA	no	sampled $Q \leftarrow T^{\pi_\varepsilon} Q$ (on-policy)
	Q-learning	no	sampled value iteration $Q \leftarrow BQ$ (off-policy)

Finally, **cliff walking** shows where on-policy and off-policy give different answers.

Known model

The Bellman equation for the toy MDP

Value of a state = expected (reward of the next step + $\gamma\times$ value of the next state):

$$V^\pi(s) = \sum_{s'} p(s' \mid s, \pi(s)) \left[R(s, \pi(s), s') + \gamma V^\pi(s') \right], \qquad V^\pi(G) = 0.$$

State A, action go:

$$\begin{aligned} V(A) &= 0.8\,[0 + \gamma V(B)] + 0.2\,[0 + \gamma V(A)] \\ &= 0.72\,V(B) + 0.18\,V(A) \end{aligned}$$

State B, action cash:

$$\begin{aligned} V(B) &= 0.7\,[20 + \gamma \cdot 0] + 0.3\,[0 + \gamma V(A)] \\ &= 14 + 0.27\,V(A) \end{aligned}$$

The transition $B \to G$ (probability 0.7) contributes only its **reward** $0.7 \cdot 20 = 14$: the value term $\gamma V(G)$ is 0.

Solve: $0.82\,V(A) = 0.72\,(14 + 0.27\,V(A)) \;\Rightarrow\; V(A) = \dfrac{10.08}{0.6256} = \mathbf{16.11}$, $V(B) = \mathbf{18.35}$.

The same equation in matrix form

With all three states (rows: from; columns: to A, B, G):

$$\bar{P}^{\pi} = \begin{pmatrix} 0.2 & 0.8 & 0 \\ 0.3 & 0 & 0.7 \\ 0 & 0 & 1 \end{pmatrix}, \qquad r^{\pi}(s) = \sum_{s'} p(s' \mid s, \pi(s))\, R(s, \pi(s), s') = \begin{pmatrix} 0 \\ 14 \\ 0 \end{pmatrix}.$$

Since $V(G) = 0$, the **column** G multiplies 0 and the **row** G is not needed. Keep only $\{A, B\}$:

$$P^{\pi} = \begin{pmatrix} 0.2 & 0.8 \\ 0.3 & 0 \end{pmatrix}, \qquad r^{\pi} = \begin{pmatrix} 0 \\ 14 \end{pmatrix}.$$

The 0 in row B is $p(B \mid B, \text{cash})$: cash never leads back to B. The 0.7 goes to G and survives only in $r^{\pi}(B) = 0.7 \cdot 20$; row B sums to 0.3, the missing mass is the probability of finishing.

$$V^{\pi} \;=\; r^{\pi} \;+\; \gamma P^{\pi} V^{\pi} \qquad \Longrightarrow \qquad V^{\pi} \;=\; (I \;-\; \gamma P^{\pi})^{-1} r^{\pi}$$

Row A reads $V(A) = 0 + 0.9\,(0.2 V(A) + 0.8 V(B))$, as on the previous slide. Cost $O(|S|^3)$.

Simple iterations

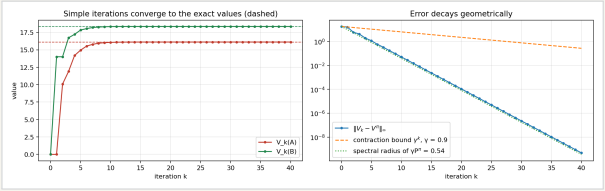
$$V_{k+1} = T^\pi V_k := r^\pi + \gamma P^\pi V_k, \quad V_0 = 0.$$

T^π is a γ-contraction, since the rows of P^π are ≥ 0 and sum to ≤ 1:

$$\|T^\pi V - T^\pi U\|_\infty \leq \gamma \|V - U\|_\infty.$$

Banach: unique fixed point V^π, and $\|V_k - V^\pi\|_\infty \leq \gamma^k \|V_0 - V^\pi\|_\infty$.

Actual rate: spectral radius of $\gamma P^\pi = 0.54 < 0.9$, because every step may end the episode.



TD(0) below is exactly this iteration with $P^\pi V_k$ replaced by **one sampled transition**.

Unknown model: evaluation

Now forget the model

The learner may only call `env_step(s, a, rng)`: sample $s' \sim p(\cdot \mid s, a)$, return (s', r). It never reads P.

Three episodes under $\pi = (\text{go}, \text{cash})$:

path	rewards
$A \to B \to A \to B \to A \to B \to G$	$0, 0, 0, 0, 0, 20$
$A \to A \to B \to G$	$0, 0, 20$
$A \to B \to G$	$0, 20$

All randomness comes from the transitions (the policy is deterministic). The return from A is $G = 20\gamma^{K-1}$, K = number of steps: mean 16.11, standard deviation about 2.65.

$V^\pi(s) = \mathbb{E}[G_t \mid s_t = s]$ is a **mean of a random variable we can sample**.

Monte Carlo: average the returns

$$V(s_t) \leftarrow V(s_t) + \alpha\big(G_t - V(s_t)\big), \qquad \alpha = \tfrac{1}{N(s)} \;\Rightarrow\; \text{running sample mean.}$$

- **Target** $X = G_t$, mean $\mathbb{E}[G_t \mid s_t = s] = V^\pi(s)$: a **fixed** number. Unbiased; LLN gives convergence, error $\approx \sigma_s/\sqrt{N(s)}$.
- **Returns backwards:** $G_t = r_{t+1} + \gamma G_{t+1}$ from $G = 0$ at the terminal state.
  Episode $A \to A \to B \to A \to B \to G$: $G = 13.12,\ 14.58,\ 16.2,\ 18,\ 20$.
- **First visit:** $V(A)$ gets 13.12, $V(B)$ gets 16.2; later visits are skipped, so episodes give independent samples.
- Updates only **after the episode ends**; the target carries the noise of the whole episode.

No model, no Markov assumption: just the law of large numbers, one state at a time.

TD(0): one sampled step of the simple iteration

MC uses the observed rest of the return G_{t+1}. TD replaces it by the **current table entry of the next state**, $V(s_{t+1})$:

$$V(s_t) \leftarrow V(s_t) + \alpha\big(Y_t - V(s_t)\big), \qquad Y_t = \underbrace{r_{t+1} + \gamma V(s_{t+1})}_{\text{TD target}}, \qquad V(G) = 0.$$

- $V(s_{t+1})$ is whatever the table holds **now** for the next state: one real step, then trust the estimate (bootstrapping).
- The update is made after every step; no need to wait for the end of the episode.
- MC: $Y_t = G_t$. TD: $Y_t = r_{t+1} + \gamma V(s_{t+1})$. Same rule $V \leftarrow V + \alpha(Y - V)$, different random target.

The TD target in the toy MDP

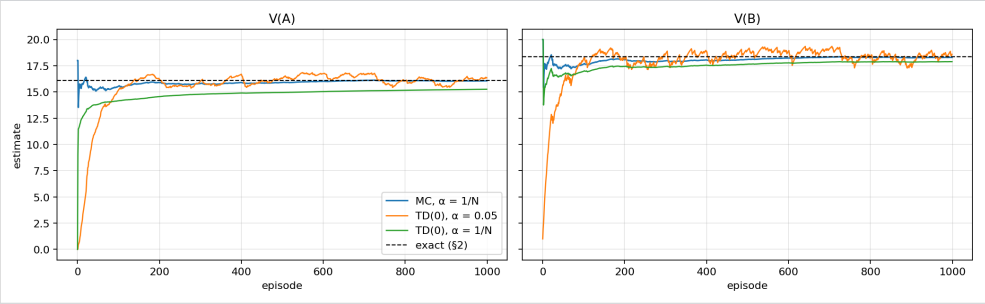
Toy MDP. Y is random because the next state is random:

update in	transition (next state s')	prob.	target $Y = r + \gamma V(s')$
$s_t = A$ (go)	$A \to B$, $r = 0$	0.8	$0.9\,V(B)$
	$A \to A$, $r = 0$ (slip: s' is A again)	0.2	$0.9\,V(A)$
$s_t = B$ (cash)	$B \to G$, $r = 20$	0.7	$20 + 0.9\,V(G) = 20$
	$B \to A$, $r = 0$	0.3	$0.9\,V(A)$

Mean of the target: $\mathbb{E}[Y \mid A] = 0.72V(B) + 0.18V(A)$, $\mathbb{E}[Y \mid B] = 14 + 0.27V(A)$: exactly the right-hand sides of the Bellman equations, $(T^\pi V)(s)$. Setting $\mathbb{E}[Y \mid s] = V(s)$ gives $(16.11, 18.35) = V^\pi$.

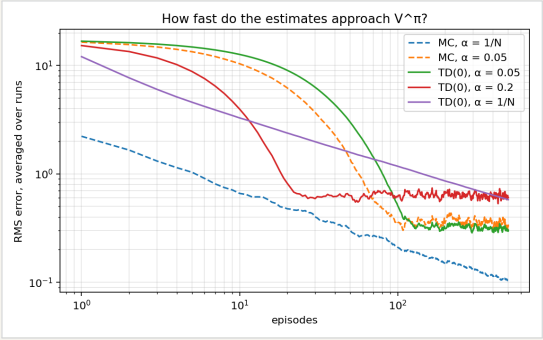
MC: Robbins–Monro for a fixed **mean** $V^\pi(s)$. TD: for the **fixed point** $V \quad = \quad T^\pi V$ (the mean moves with V).

One run of each method



- **MC,** $\alpha = 1/N$: running average of unbiased samples; noisy at first, then settles.
- **TD, constant** α: starts from 0, then keeps fluctuating around V^π (variance of order α); reacts quickly.
- **TD,** $\alpha = 1/N$: converges, but slowly: early targets used $V(s') \approx 0$ and keep weight $1/N$.

Averaged over 100 runs



RMS error after 500 episodes:

MC, $\alpha = 1/N$	**0.11**
MC, $\alpha = 0.05$	0.34
TD, $\alpha = 0.05$	0.30
TD, $\alpha = 0.2$	0.60
TD, $\alpha = 1/N$	0.58

- **MC wins here**: episodes last ~ 3 steps, little variance to remove; error $\propto 1/\sqrt{n}$.
- Constant α: error floor $\sim \sqrt{\alpha}$.
- TD's advantage needs long, noisy episodes or tasks that never end.

What each method does with a fixed data set

Collect 20 episodes once and replay them until the estimates stop changing (**batch** updating).

- **Batch MC** $\rightarrow$ sample average of the observed first-visit returns.
- **Batch TD** $\rightarrow$ the **certainty-equivalence** estimate: fit $\hat{P}^\pi$ (transition frequencies), $\hat{r}^\pi$ (average rewards), then solve $\hat{V} = (I - \gamma \hat{P}^\pi)^{-1} \hat{r}^\pi$.

20 episodes	$V(A)$	$V(B)$
batch MC	16.30	18.55
batch TD	16.35	18.49
fitted model $(I - \gamma \hat{P})^{-1} \hat{r}$	16.35	18.49
exact	16.11	18.35

TD does model-based evaluation on the maximum-likelihood model, without storing $\hat{P}$.

Every visit to B informs $V(A)$ through $\hat{P}(A \rightarrow B)$. Not Markov $\Rightarrow$ wrong $\hat{P}$, and MC is safer.

MC vs. TD

	Monte Carlo	TD(0)
target	G_t	$r_{t+1} + \gamma V(s_{t+1})$
random through	the rest of the episode	one step
mean of target	$V^\pi(s)$, fixed	$(T^\pi V)(s)$, moving
bias	none	yes, vanishes as $V \to V^\pi$
updates	end of episode	every step
Markov property	not used	used
converges by	law of large numbers	contraction + Robbins–Monro
batch limit	sample means of returns	certainty equivalence

Unknown model: control

From evaluation to control: action values

With V, improvement needs $p(s'|s,a)$; with Q it does not: $\pi'(s) = \arg\max_a Q(s,a)$.

$$Q^\pi(s,a) = \sum_{s'} p(s'|s,a)\Big[R + \gamma \sum_{a'} \pi(a'|s') Q^\pi(s',a')\Big] = (T^\pi Q^\pi)(s,a),$$

$$Q^*(s,a) = \sum_{s'} p(s'|s,a)\Big[R + \gamma \max_{a'} Q^*(s',a')\Big] = (BQ^*)(s,a).$$

Both are γ-contractions. Learners explore ε-**greedily**, $\varepsilon = 0.3$ (greedy action w.p. 0.85).

exact (with the model)	$Q(A,\text{stay})$	$Q(A,\text{go})$	$Q(B,\text{cash})$	$Q(B,\text{back})$
Q^*	15.50	16.11	18.35	14.50
Q^{π_ε}, $\varepsilon = 0.3$	14.68	15.29	18.10	13.68

$Q^{\pi_\varepsilon} < Q^*$: exploring sometimes stays or goes back. Same greedy policy (go, cash);
$Q^*(A,\text{go}) = 16.11 = V^\pi(A)$, so the policy we evaluated is optimal.

SARSA, Expected SARSA, Q-learning

All three act ε-greedily and update one entry per step:
$Q(s,a) \leftarrow Q(s,a) + \alpha\,\big(\text{target} - Q(s,a)\big)$.

method	target	random through	fixed point
SARSA	$r + \gamma Q(s',a')$, a' actually taken	s' and a'	Q^{π_ε} (on-policy)
Expected SARSA	$r + \gamma \sum_{a'} \pi_\varepsilon(a'\|s') Q(s',a')$	s' only	Q^{π_ε}
Q-learning	$r + \gamma \max_{a'} Q(s',a')$	s' only	Q^* (off-policy)

- At a terminal s' the target is just r.
- SARSA chooses a' **before** the update and then takes it: one draw serves to act and to form the target.
- Expected values: SARSA's target has mean $(T^{\pi_\varepsilon} Q)(s,a)$; Q-learning's has mean $(BQ)(s,a)$.

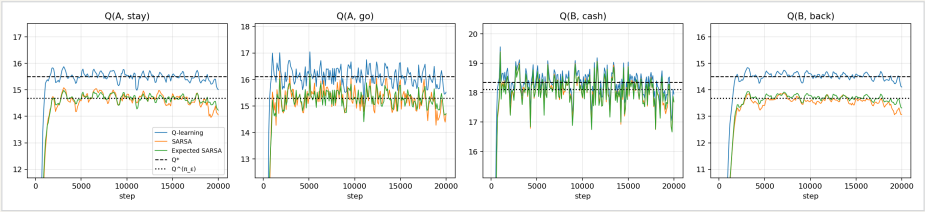
One update, variable by variable

$$Q_{k+1}(s_t, a_t) = Q_k(s_t, a_t) + \alpha\big[Y - Q_k(s_t, a_t)\big], \qquad Y = \begin{cases} r_{t+1} + \gamma\, Q_k(s_{t+1}, a_{t+1}) & \text{SARSA} \\ r_{t+1} + \gamma \max_{a'} Q_k(s_{t+1}, a') & \text{Q-learning} \end{cases}$$

symbol	kind	where it comes from
$s_t,\ a_t$	known	where the agent is and what it just did
$r_{t+1},\ s_{t+1}$	**sampled**	environment, $s_{t+1} \sim p(\cdot \mid s_t, a_t)$ (replaces $\sum_{s'}$)
a_{t+1} (SARSA)	**sampled**	ε-greedy choice from the **old** Q_k (replaces $\sum_{a'} \pi(a'\|s')$)
$Q_k(s_t, a_t)$	**old**	the entry being corrected
$Q_k(s_{t+1}, a_{t+1})$, $\max_{a'} Q_k(s_{t+1}, a')$	**old**	replace unknown Q^π, Q^* (bootstrapping)
$Q_{k+1}(s_t, a_t)$	**new**	the only entry that changes

The target uses only the old table; then one entry is overwritten, all others keep their values. "Old" can be stale: $Q_k(s_{t+1}, \cdot)$ is what was written at the last visit to s_{t+1} (early on, 0). At the terminal state $Q = 0$, so $Y = r_{t+1}$.

Two fixed points



20,000 steps, $\alpha = 0.05$, mean of 10 runs	$Q(A, \text{stay})$	$Q(A, \text{go})$	$Q(B, \text{cash})$	$Q(B, \text{back})$
exact Q^*	15.50	16.11	18.35	14.50
exact Q^{π_ε}	14.68	15.29	18.10	13.68
Q-learning	15.34	15.92	18.26	14.45
SARSA	14.43	15.05	17.98	13.46
Expected SARSA	14.53	15.08	17.97	13.62

Q-learning $\to Q^*$; SARSA and Expected SARSA $\to Q^{\pi_\varepsilon}$, the value of the policy they actually follow. All find (go, cash).

The three targets as random variables

Freeze Q at its fixed point and sample the target for (A, go) $(r = 0)$:

method	values of the target (probability)	mean	std
SARSA	16.29 (0.68), 12.31 (0.12), 13.76 (0.17), 13.21 (0.03)	15.29	1.51
Expected SARSA	15.70 (0.8), 13.68 (0.2)	15.29	0.81
Q-learning	16.52 (0.8), 14.50 (0.2)	16.11	0.81

SARSA: $s' \in \{B, A\} \times$ next action greedy/other; the others: $s' = B$ / $s' = A$.

- Each mean is what its fixed-point equation requires: $Q^{\pi_\varepsilon}(A, \text{go}) = 15.29$, $Q^*(A, \text{go}) = 16.11$.
- Expected SARSA averages the next action out exactly: same mean as SARSA, **3.5 times smaller variance**.
- Q-learning also has only environment noise, but uses the **greedy** next action: hence Q^*.

Off-policy in action: learning from random behavior

Let the agent act **uniformly at random** ($\varepsilon = 1$), 10 runs:

	$Q(A, \text{stay})$	$Q(A, \text{go})$	$Q(B, \text{cash})$	$Q(B, \text{back})$
exact Q^*	15.50	16.11	18.35	14.50
exact Q of the random policy	11.90	12.32	17.27	10.90
Q-learning	15.28	15.79	18.16	14.37
SARSA	11.64	12.01	16.99	10.62

- Q-learning's target depends on the data only through (s, a) and $s' \sim p(\cdot|s, a)$: the behavior decides **which pairs** are updated, not the mean of the target. No importance weights needed.
- It learned Q^* from data in which the agent never tried to act well. This is what makes experience replay (DQN) possible.
- SARSA answered a different question: how good is random behavior.

SARSA reaches Q^* if exploration fades (GLIE)

With fixed ε, SARSA learns Q^{π_ε}. **GLIE**: $\varepsilon_t(s) \to 0$ but $\sum_t \varepsilon_t(s) = \infty$ (greedy in the limit, infinite exploration). Here $\varepsilon_t(s) = 1/\sqrt{n_t(s)}$, $n_t(s) =$ visits to s.

	$Q(A, \text{stay})$	$Q(A, \text{go})$	$Q(B, \text{cash})$	$Q(B, \text{back})$
exact Q^*	15.50	16.11	18.35	14.50
exact Q^{π_ε}, $\varepsilon = 0.3$	14.68	15.29	18.10	13.68
SARSA, fixed $\varepsilon = 0.3$	14.43	15.05	17.98	13.46
SARSA, GLIE (50,000 steps)	15.28	**16.11**	**18.11**	14.32

The greedy actions approach Q^*. The non-greedy actions are tried less and less, so their estimates lag: GLIE converges, but slowly, exactly for the actions it stops exploring.

Cliff walking

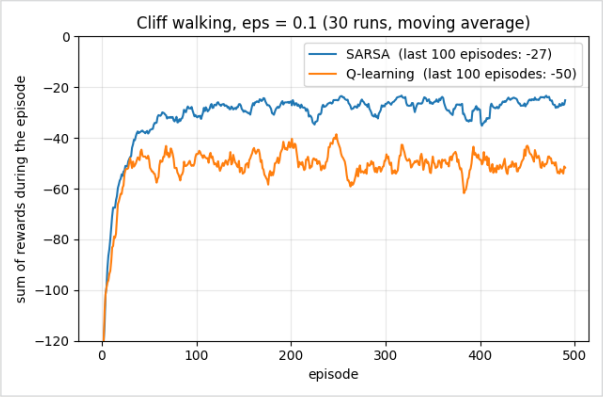
Cliff walking: where on-policy and off-policy differ

Gymnasium `CliffWalking`: 4×12 grid, start bottom left, goal bottom right, cliff in between.

- every step -1;
- cliff: -100, back to the start;
- actions: up, right, down, left.

$\varepsilon = 0.1$, $\alpha = 0.5$, $\gamma = 1$, 500 episodes, 30 runs.

Predict: which algorithm collects more reward **while learning**? Which path will each greedy policy take?



The learned greedy paths



SARSA: safe path along the top, 17 steps



Q-learning: along the edge, 13 steps (optimal)

- **Q-learning** learns Q^*, whose greedy path hugs the cliff. But it **behaves** ε-greedily: on the edge a random step goes down with probability $\varepsilon/4$, costing -100. About $\mathbf{-50}$ per episode while learning.
- **SARSA** learns the value of the ε-greedy policy it actually follows, accidental falls included. Cells next to the cliff look bad, so it keeps a safe distance: about $\mathbf{-27}$ per episode.
- Greedy runs ($\varepsilon = 0$): neither falls. As $\varepsilon \to 0$, both converge to the optimal path.

(Greedy policies of the Q-tables averaged over the 30 runs.)

Which one should we use?

Q-learning learns the best way to act. SARSA learns the best way to act **given that you keep exploring**.

Prefer Q-learning (off-policy)

- learning in a simulator, then acting greedily;
- reusing data from old policies, humans, a replay buffer;
- you want Q^* without tuning a GLIE schedule.

Prefer SARSA (on-policy)

- learning online, where mistakes while exploring are costly (a real robot, the cliff);
- with function approximation, on-policy TD is more stable.

Expected SARSA: lower variance than SARSA; with a greedy target policy it is exactly Q-learning.

Summary

Summary

method	update / target	model	learns
inverse matrix	$V = (I - \gamma P^\pi)^{-1} r^\pi$	yes	V^π exactly
simple iterations	$V \leftarrow r^\pi + \gamma P^\pi V$	yes	V^π (contraction)
Monte Carlo	G_t	no	V^π; unbiased, high variance
TD(0)	$r + \gamma V(s')$	no	V^π; biased early, low variance
SARSA	$r + \gamma Q(s', a')$	no	Q^{π_ε}; on-policy
Expected SARSA	$r + \gamma \sum_{a'} \pi_\varepsilon(a'\|s') Q(s', a')$	no	Q^{π_ε}; lower variance
Q-learning	$r + \gamma \max_{a'} Q(s', a')$	no	Q^*; off-policy

One update rule, $x \leftarrow x + \alpha(\text{target} - x)$; the methods differ only in the target.

Questions for the notebook

1. Change π to (stay, cash). What happens to the simple iterations, to MC (does an episode ever end?) and to TD?
2. Set the success probability of *cash* to 0.3. Episodes get longer: does the gap between MC and TD grow?
3. Batch updating with 5 episodes instead of 20: which estimate is closer to the exact values, and why?
4. Change the reward of *stay* from 1 to 1.6. Do the greedy policies of Q^* and Q^{π_ε} still agree?
5. Cliff walking: Q-learning's greedy path is shorter, so why is its reward during learning worse? Which algorithm would you trust to drive a real robot while it is still learning?

Notebook: `notebook14.ipynb`.